

Materiale utile laborator PA

1. In caz ca vreti sa reluati elementele de sintaxa ale limbajului Python, puteti sa urmariti tutorialul de pe <https://www.w3schools.com/python/>. In partea stanga aveti un meniu pentru navigare rapida (de ex: variabile/string-uri/valori de tip bool etc.). De asemenea puteti face quiz-uri pentru testarea cunostintelor. (PS: Nu vom studia la laborator toate notiunile prezentate pe acest site, de aceea este bine sa invatati mai intai notiunile predate la curs si la seminar).
2. Avand in vedere ca la laborator vom implementa probleme de algoritmica, aveti mai jos o lista de notiuni care sunt folositoare atunci cand rezolvati exercitiile.
3. In caz ca o notiune nu este clara, este indicat sa o executati in REPL (Windows + R => cmd [Enter] => python [Enter]; apare >>>). Daca tot nu este evident cum functioneaza puteti sa o cautati in documentatia limbajului Python (<https://docs.python.org/3/> => dreapta sus - Quick search).

Operatii elementare:

1. Variabilele sunt asemanatoare cu **pointerii/referintele** din C/C++ (ceea ce inseamna ca arata catre o valoare):
 - `a = 12` `# a arata catre locatia din memorie care contine numarul 12`
 - `a = "ana"` `# a "uita" ca arata catre valoarea 12 si acum arata catre "ana"`
 - `# am pierdut legatura catre 12 si nu o mai putem accesa... trebuia sa facem o copie inainte de a asigna alta valoare daca voiam sa o pastram.`
2. Control flow (if/while/for):
 - nu trebuie sa mai puneti parantezele pentru conditii
 - **trebuie** sa puneti : dupa instructiuni si sa **aliniati** restul instructiunilor
 - `for i in range(n):` `# for (int i = 0; i < n; i++)` ← mai mic **strict**
 - `for i in range(in, sf):` `# for (int i = in; i < sf; i++)` ← mai mic **strict**
 - `for i in range(0, n, p):` `# for (int i = 0; i < n; i += p)` ← mai mic **strict**
3. Conversie la un tip de date existent (int, string, bool etc.):
 - `int(string)`
 - ex: `n = int('123');` `s = str(12);` `b = bool(0);` `b = bool('0')`
 - unii constructori primesc mai multe argumente; ex: `int("101", base=2)` `== 5`
4. Limbajul Python este strong typed ceea ce inseamna ca nu face conversie automat intre tipuri (exista cateva exceptii - comoditate):
 - `a = "ana" + 3` `# eroare nu se poate aduna un string cu un int`
 - in schimb, in principiu se poate face conversie de la orice tip la string:
 - `a = "ana" + str(3)` `# functioneaza: "ana" + "3" = "ana3"`

Operatii pe biti

1. Operatiile elementare (tabel de adevar):

- https://en.wikipedia.org/wiki/Boolean_algebra#Basic_operations &, |, ~
- https://en.wikipedia.org/wiki/Boolean_algebra#Secondary_operations
->, ^, ≡

2. Declaratie numar intr-o baza a (a = b - binar, o - octal, a = x - hexazecimal):

- 0**a**1001 (in loc de **a** inlocuiti cu una din literele de mai sus)
- ex: n = 0b1110; print(n) => 14
- cand specificati o baza puteti pune zero-uri nesemnificative (0b0010 = 2)

3. Conversie numar din baza 10 in baza (2, 8, 16) in string:

- bin(n), oct(n), hex(n)
- ex: bin(11) = '0b1011' # il primiti ca string
- bin(11)[2:] == '1011' # tot ca string, dar fara 0b din fata

4. Numerele negative sunt salvate in 2's complement ceea ce inseamna ca numerele negative nu vor fi afisate "corect" cand sunt convertite in baza 2.

Pentru numerele cu semn (+/-) primul bit indica semnul (+ = 0, - = 1) :

- pentru numere pe 4 biti, o metoda simpla este sa setam primul bit la 0 daca numarul este pozitiv si la 1 daca este negativ si sa pastram restul bitilor; de ex: 0b**0**110 = **+6** si 0b**1**011 = **-3**; bit-ul ingrosat nu se foloseste pentru numar ci doar pentru semn => putem salva astfel numere intre -7 si 7 in loc de 0 si 15.
- 2's complement = nu este eficienta reprezentarea de mai sus cand vrem sa calculam $a + (-a) = 0$, deci $5 + -5 = 0$, deci $0101 + 1101 = 0$; 5-ul il pastram dar am vrea atunci cand adunam -5 ca rezultatul sa fie **10000** (pe **5** biti!)
- $-5 = 1011$; **intrebare**: cum putem ajunge de 5 (0101) la -5 (1011)?
- de exemplu operatia **not**: $\sim 5 = -6$; (not-ul inversa toti bitii, pentru 101 avem 010 cand inversam; putem testa acest lucru: $(-5) \& (1 \ll 1) == 2$, deci not-ul functioneaza si in plus: $0110 + 1010 = 10000$ (tot pe **5** biti!))

Operatii string-uri:

1. Operatii uzuale siruri de caractere:

- `s = "Ana are mere"`
- `for c in s: # parcurgere sir`
`print(c, end=" ")`
- dimensiunea unui sir de caractere: `len(s) == 12`
- concatenare: `s + "!" == "Ana are mere!"`
- indexare: `s[0] == 'A'` si `s[-1] == 'e'`
- slice-uri:
 - i. `s[1:4] = "na "` # sir-ul s de la indexul 1 pana la 4, exclusiv
 - ii. `s[:4] = "Ana"` # nu specificam inceputul => 0:4
 - iii. `s[::-1] = 'erem era anA'` # Al treilea parametru reprezinta pasul

2. String-urile sunt tipuri de date de tip immutable ceea ce inseamna ca nu se mai pot modifica dupa ce au fost initializate (`s + "1"` construiesc un **nou** string); daca vrem sa modificam un string il redeclaram: `s = s + "1"`.

3. Functii de conversie intre un **caracter**(string format dintr-un singur caracter) si ASCII

- `ord('a') = 97`
- `chr(98) = 'b'`

4. Ordine relativa intre caractere:

- vrem sa stim numarul de ordine al literei c (c este a treia litera din alfabet - sau a **doua** daca indexam de la 0):
 - i. `nr_c = ord('c')`
 - ii. `nr_a = ord('a')`
 - iii. `nr_ordine = nr_c - nr_a # 2`

Operatii liste

1. Operatii uzuale pe liste:

- a. lista vida: `xs = []`
- b. adaugare element: `xs.append(123)`
- c. extragere ultimul el: `x = xs.pop()`
- d. extragere el de pe **idx**: `x = xs.pop(idx)`
- e. parcurgere lista:

```
for x in xs:  
    print(x)
```
- f. dimensiune lista: `len(xs)`
- g. slice-uri (ca la string-uri)

2. Tineti minte de la **Operatii elementare 1**) variabilele sunt referinte:

- a. `xs = [1, 2, 3]`
- b. `ys = xs`
- c. `ys.append(4)`
- d. `print(ys) # [1, 2, 3, 4]`
- e. `print(xs) # [1, 2, 3, 4] ! important !`
- f. pentru copie folositi slice-uri: `ys = xs[:]`

3. ... urmeaza ...